# Données avancées avec TypeScript

Tous droits réservés à STUDI - Reproduction interdite

# Table des matières

**I - Contexte** **3**

**II - Les tableaux, les tuples et les énumérations** **4**

A. Les tableaux, les tuples et les énumérations ..... 4

B. Exercice : Quiz ..... 8

**III - Les fonctions (functions)** **10**

A. Les fonctions (functions) ..... 10

B. Exercice : Quiz ..... 14

**IV - Le casting** **17**

A. Le casting ..... 17

B. Exercice : Quiz ..... 18

**V - Essentiel** **21**

**VI - Auto-évaluation** **22**

A. Exercice ..... 22

B. Test ..... 22

**Solutions des exercices** **24**



Tous droits réservés à STUDI - Reproduction interdite

# I Contexte

**Durée : 1 heure**

**Prérequis : les bases en JavaScript**

Contexte

Dans ce cours, nous allons découvrir et travailler avec des types de données de référence (types de données objets) tels que les tableaux, les énumérations, les *tuples*, les classes et les fonctions.

Les types objets offrent des fonctionnalités plus avancées que les types primitifs. Les tableaux permettent de stocker plusieurs valeurs de même type, tandis que les énumérations fournissent une liste de valeurs nommées qui peuvent être utilisées pour faciliter la lecture et la compréhension du code. Les tuples, quant à eux, permettent de stocker des ensembles de valeurs de différents types dans un ordre spécifique. Les fonctions peuvent également être définies avec des types de paramètres et de retours spécifiques, ainsi qu'avec des fonctions anonymes et des fonctions fléchées pour une syntaxe plus concise.

Avec toutes ces fonctionnalités, TypeScript permet aux développeurs de créer des applications plus robustes et maintenables.

Tous droits réservés à STUDI - Reproduction interdite

# II Les tableaux, les tuples et les énumérations

## A. Les tableaux, les tuples et les énumérations

### Les tableaux (arrays)

Az Définition

Les tableaux sont des structures de données qui permettent de stocker une collection d'éléments du même type. Les tableaux en TypeScript sont similaires aux tableaux en JavaScript, mais TypeScript offre un typage fort qui permet de s'assurer que les éléments du tableau sont du bon type.

### Déclaration et initialisation de tableaux

En TypeScript, les tableaux sont déclarés en spécifiant le type de données des éléments du tableau suivi de `[]`. Par exemple, un tableau contenant des nombres entiers peut être déclaré de la manière suivante :

```
1 let myArray : number[] = [1, 2, 3, 4, 5];
```

Notez que le type de données des éléments du tableau est `number`, suivi de `[]` (`number[]`) pour indiquer que c'est un tableau de nombres, contenant les éléments 1, 2, 3, 4 et 5.

TypeScript permet également de définir des tableaux à plusieurs dimensions en utilisant des tableaux imbriqués :

```
1 let myMatrix: number[][] = [[1, 2], [3, 4]];
```

### Accès aux éléments du tableau

Les éléments du tableau peuvent être accédés en utilisant des index numériques, comme ceci :

```
1 let myArray: number[] = [1, 2, 3];
2 let secondElement : number = myArray[1]; // Résultat 2
```

Ici, nous accédons au deuxième élément du tableau (2) en utilisant l'index 1.

### Tableaux de types génériques

Il est également possible de définir des tableaux de types génériques en TypeScript, ce qui permet de définir des tableaux qui contiennent des éléments de types différents. Par exemple, un tableau qui contient une chaîne de caractères et un nombre peut être défini de la manière suivante :

```
1 let myArray : Array<string | number> = ['Hello', 20];
```

Notez que nous avons utilisé le type `Array<string | number>` au lieu de `string[]` ou `number[]` pour indiquer que le tableau peut contenir des éléments de type string ou number.

### Tableaux utilisés pour une application de gestion des tâches

Exemple

Supposons que nous ayons une application qui doit stocker une liste de tâches à faire pour un utilisateur. Nous pouvons utiliser un tableau pour stocker ces tâches :

```
1 let tasks : string[] = ["Faire les courses", "Nettoyer la maison", "Rendre
  visite à un ami"];
```


Tous droits réservés à STUDI - Reproduction interdite

Nous pouvons ensuite parcourir ce tableau pour afficher chaque tâche à faire :

```
for (let i = 0; i < tasks.length; i++) {
  console.log(`Tâche n°${i + 1}: ${tasks[i]}`);
}
```

Si nous avons besoin d'une collection de valeurs homogènes, on utilise un tableau générique. Mais si nous avons besoin de stocker une séquence de valeurs hétérogènes de manière ordonnée et que le nombre d'éléments est fixe, on utilise un tuple.

**Les tuples** Az Définition

Les tuples sont des types de données spécifiques à TypeScript, qui permettent de stocker une séquence d'éléments de même type ou de types différents. Les tuples peuvent être utilisés pour représenter des données structurées comme une paire de coordonnées (x, y), un enregistrement de données avec différents champs (nom, âge, adresse, etc.).

## Déclaration de tuples

Les tuples sont déclarés en spécifiant les types de données des éléments à l'intérieur de crochets [], séparés par des virgules. Par exemple, un tuple qui contient une chaîne de caractères et un nombre peut être déclaré de la manière suivante :

```
let myTuple: [string, number];
```

## Initialisation de tuples

Les tuples peuvent être initialisés en affectant des valeurs correspondantes aux éléments de tuple. La syntaxe est similaire à celle utilisée pour les tableaux :

```
let myTuple: [string, number] = ['Hello', 20];
```

Ici, `myTuple` est un tuple contenant une chaîne de caractères (`string`) et un nombre (`number`). Notez que le nombre d'éléments dans le tuple doit correspondre au nombre de types de données spécifiés dans la déclaration de tuple. Dans l'exemple ci-dessus, nous avons spécifié 2 types de données (`string` et `number`) dans la déclaration de tuple, donc la longueur de notre tuple doit être 2.

## Accès aux éléments de tuples

On peut accéder aux éléments d'un tuple en utilisant des index numériques. Par exemple, pour accéder au premier élément d'un tuple myTuple, nous utiliserions la syntaxe `myTuple[0]`. Pour accéder au deuxième élément, nous utiliserions `myTuple[1]`, et ainsi de suite.

```
let myTuple : [string, number] = ['Hello', 20];
let myString : string = myTuple[0]; // Hello
let myNumber : number = myTuple[1]; // 20
```

Notez que TypeScript permet également de décomposer un tuple en plusieurs variables séparées, ce qui peut rendre le code plus lisible :

```
let myTuple: [string, number] = ['Hello', 20];
let [myString, myNumber] = myTuple;
```

Ici, nous avons décomposé le tuple myTuple en 2 variables séparées, `myString` et `myNumber`, qui contiennent respectivement la chaîne de caractères `'Hello'` et le nombre 20.

Tous droits réservés à STUDI - Reproduction interdite

### Tuples utilisés pour stocker des informations

Exemple

Supposons maintenant que nous avons besoin de stocker à la fois le nom et l'adresse e-mail d'un utilisateur. Nous pouvons utiliser un tuple pour stocker ces informations :

```
let user: [string, string] = ["Alice", "alice@example.com"];
```

Nous pouvons ensuite utiliser ces informations pour envoyer un e-mail de bienvenue à l'utilisateur :

```
console.log(`Bienvenue, ${user[0]} ! Un e-mail de bienvenue a été envoyé à ${user[1]}.`);
```

## Tableaux de tuples

Il est également possible de définir des tableaux de tuples en TypeScript. Pour déclarer un tableau de tuples, nous utilisons la syntaxe `[]` pour déclarer un tableau, et la syntaxe de tuple pour déclarer chaque élément du tableau.

TypeScript permet également de définir des tableaux de tuples :

```
let myArray: [string, number][] = [['Hello', 20], ['World', 99]];
```

Notez que la longueur de chaque tuple doit correspondre à la longueur de la déclaration de tuple de tableau. Dans l'exemple ci-dessus, nous avons déclaré un tableau de tuples de type `[string, number][]`, donc chaque élément du tableau doit être un tuple avec exactement 2 éléments : une chaîne de caractères et un nombre.

### Tableaux de tuples utilisés pour stocker des informations

Exemple

Supposons que nous ayons besoin de stocker des informations sur plusieurs utilisateurs, y compris leur nom, leur adresse e-mail et leur numéro de téléphone. Nous pouvons utiliser un tableau de tuples pour stocker ces informations pour chaque utilisateur :

```
let users: [string, string, string][] = [
  ["Alice", "alice@example.com", "123-456-7890"],
  ["Bob", "bob@example.com", "987-654-3210"],
  ["Charlie", "charlie@example.com", "555-555-5555"]
];
```

Nous pouvons ensuite parcourir ce tableau pour afficher les informations de chaque utilisateur :

```
for (let i = 0; i < users.length; i++) {
  console.log(`Nom: ${users[i][0]}, Email: ${users[i][1]}, Téléphone: ${users[i][2]}`);
}
```

Le code sera plus lisible si nous utilisons des interfaces pour décrire les informations de chaque utilisateur. Pour avoir une idée, voici un exemple :

```
interface User {
  name: string;
  email: string;
  phone: string;
}

let users: User[] = [
  { name: "Alice", email: "alice@example.com", phone: "123-456-7890" },
  { name: "Bob", email: "bob@example.com", phone: "987-654-3210" },
  { name: "Charlie", email: "charlie@example.com", phone: "555-555-5555" }
];
```


Tous droits réservés à STUDI - Reproduction interdite

Nous pouvons ensuite parcourir ce tableau pour afficher les informations de chaque utilisateur de manière plus claire :

```
for (let i = 0; i < users.length; i++) {
  console.log(`Nom: ${users[i].name}, Email: ${users[i].email}, Téléphone: ${users[i].phone}`);
}
```

### Les énumérations (enums)

Az Définition

Les énumérations sont aussi des types de données spécifiques à TypeScript, elles n'existent pas en JavaScript. Elles permettent de définir un ensemble de valeurs nommées. Les énumérations sont souvent utilisées pour représenter des constantes qui ont une signification particulière.

## Déclaration d'une énumération

Une énumération qui représente les jours de la semaine peut être définie de la manière suivante :

```
enum DaysOfWeek {
  Monday,
  Tuesday,
  Wednesday,
  Thursday,
  Friday,
  Saturday,
  Sunday
}
```

Dans cet exemple, nous avons défini une énumération `DaysOfWeek` qui contient 7 valeurs numériques nommées. Les valeurs numériques sont automatiquement assignées par défaut, commençant à 0 et s'incrémentant de 1 pour chaque valeur suivante. Ainsi, `DaysOfWeek.Monday` a la valeur 0, `DaysOfWeek.Tuesday` a la valeur 1, et ainsi de suite.

## Utilisation d'une énumération

Les valeurs d'une énumération peuvent être utilisées de la même manière que les constantes en JavaScript. Par exemple, pour utiliser `DaysOfWeek.Monday`, nous pouvons l'écrire simplement comme ceci :

```
let today : DaysOfWeek = DaysOfWeek.Monday;
```

Ici, nous avons déclaré une variable `today` de type `DaysOfWeek`, qui contient la valeur `DaysOfWeek.Monday`.

## Assignation de valeurs personnalisées

Il est également possible de spécifier des valeurs numériques personnalisées pour chaque élément d'une énumération :

```
enum DaysOfWeek {
  Monday = 1,
  Tuesday,
  Wednesday,
  Thursday,
  Friday,
}
```

Tous droits réservés à STUDI - Reproduction interdite

### Énumérations utilisées pour une application de pizzeria

Exemple

Supposons maintenant que nous ayons besoin d'une énumération pour représenter les différents types de pizzas dans notre application de commande de pizza :

```
enum PizzaType {
  Margherita,
  Pepperoni,
  Hawaiian,
  Vegetarian
}
```

Nous pouvons utiliser cette énumération pour stocker le type de pizza commandé par un utilisateur :

```
interface Order {
  customerName: string;
  pizzaType: PizzaType;
}
let order: Order = {
  customerName: "Alice",
  pizzaType: PizzaType.Pepperoni
};
```

Nous pouvons ensuite utiliser cette information pour préparer la commande de pizza :

```
switch (order.pizzaType) {
  case PizzaType.Margherita:
    console.log(`Préparation de la pizza margherita pour ${order.customerName}...`);
    break;
  case PizzaType.Pepperoni:
    console.log(`Préparation de la pizza pepperoni pour ${order.customerName}...`);
    break;
  case PizzaType.Hawaiian:
    console.log(`Préparation de la pizza hawaiian pour ${order.customerName}...`);
    break;
  case PizzaType.Vegetarian:
    console.log(`Préparation de la pizza végétarienne pour ${order.customerName}...`);
    break;
}
```

## B. Exercice : Quiz

[solution n°1 p. 24]

### Question 1

Quelle est la syntaxe pour déclarer un tableau de nombres en TypeScript ?

- let arr: array = [1, 2, 3];
- let arr: number[] = [1, 2, 3];
- let arr = [1, 2, 3];

### Question 2

Quelle est la différence entre un tableau et un tuple en TypeScript ?


Tous droits réservés à STUDI - Reproduction interdite

Un tableau est une collection de valeurs de même type tandis qu'un tuple est une collection de valeurs de types différents/égaux

Il n'y a pas de différence significative entre un tableau et un tuple en TypeScript

Un tableau est une collection de valeurs de types différents tandis qu'un tuple est une collection de valeurs de même type

## Question 3

L'ordre des types de valeurs dans un tuple n'est pas important.

Vrai

Faux

## Question 4

Quelle est la syntaxe pour déclarer un tuple qui contient une chaîne de caractères et un nombre en TypeScript ?

let myTuple: [string, number] = [1, "Hello"];

let myTuple: [number, string] = ["Hello", 1];

let myTuple: [string, number] = ["Hello", 1];

## Question 5

Quelle est la syntaxe pour déclarer une énumération en TypeScript ?

enum Seasons{Spring, Summer, Autumn, Winter};

enum Seasons = {Spring, Summer, Autumn, Winter};

let Seasons = enum {Spring, Summer, Autumn, Winter};

Tous droits réservés à STUDI - Reproduction interdite

# III Les fonctions (functions)

## A. Les fonctions (functions)

### Les fonctions

Az Définition

Les fonctions sont des blocs de code réutilisables qui effectuent une tâche spécifique, qui peuvent être appelés à partir d'autres parties du programme. En TypeScript, nous pouvons définir une fonction en utilisant le mot-clé *function*, suivi du nom de la fonction, des paramètres entre parenthèses et du corps de la fonction entre accolades.

Exemple

Voici un exemple simple de fonction qui ajoute 2 nombres :

```
function add(a: number, b: number): number {
  return a + b;
}
```

Dans cet exemple, nous déclarons une fonction `add` qui prend 2 paramètres (a et b) de type number et qui renvoie un résultat de type `number`. À l'intérieur de la fonction, nous utilisons l'opérateur « *+* » pour ajouter les 2 paramètres et renvoyer le résultat à l'aide du mot-clé `return`.

Nous pouvons appeler cette fonction en passant 2 nombres en arguments :

```
let result = add(4, 3); // result vaut 7
```

### Paramètres optionnels

Il est possible de rendre un paramètre optionnel en le faisant suivre d'un point d'interrogation « *?* ». Dans ce cas, le paramètre ne sera pas obligatoire lors de l'appel de la fonction.

```
function greet(name?: string) {
  if (name) {
    console.log(`Bonjour, ${name} !`);
  } else {
    console.log("Bonjour !");
  }}
```

Dans cet exemple, nous déclarons une fonction `greet` qui prend un paramètre optionnel `name` de type `string`. Si le paramètre est présent, la fonction affiche un message de salutation personnalisé. Sinon, elle affiche simplement un message de salutation générique.

Nous pouvons appeler cette fonction avec ou sans argument :

```
greet(); // affiche "Bonjour !"
greet("Alice"); // affiche "Bonjour, Alice !"
```

### Paramètres par défaut

Il est également possible de définir une valeur par défaut pour un paramètre en le faisant suivre d'un signe égal « =» et de la valeur par défaut. Dans ce cas, si aucun argument n'est passé pour ce paramètre, il prendra la valeur par défaut.


Tous droits réservés à STUDI - Reproduction interdite

Exemple

```
function greet(name: string = "Monde") {
  console.log(`Bonjour, ${name} !`);
}
```

Dans cet exemple, nous déclarons une fonction `greet` qui prend un paramètre `name` de type `string` avec une valeur par défaut de « *Monde* ». Si aucun argument n'est passé pour « *name* », la fonction affiche un message de salutation générique avec le mot « *Monde* ».

Nous pouvons appeler cette fonction avec ou sans argument :

```
greet(); // affiche "Bonjour, Monde !"
greet("Alice"); // affiche "Bonjour, Alice !"
```

### Fonctions avec des types de retour complexes

Il est également possible de définir des types de retour complexes pour les fonctions en TypeScript. Par exemple, une fonction peut renvoyer un tableau, un objet ou une fonction.

### Fonctions renvoyant un tableau

Une fonction peut renvoyer un tableau en spécifiant le type `Array<T>` ou `T[]` comme type de retour, où « *T* » est le type des éléments du tableau.

Exemple

Voici un exemple de fonction qui renvoie un tableau de chaînes de caractères :

```
function getColors(): string[] {
  return ["rouge", "vert", "bleu"];
}
```

Dans cet exemple, la fonction `getColors` renvoie un tableau de chaînes de caractères. Nous utilisons le type `string[]` pour spécifier que la fonction renvoie un tableau de chaînes de caractères.

Nous pouvons appeler cette fonction et stocker le résultat dans une variable :

```
let colors = getColors(); // colors vaut ["rouge", "vert", "bleu"]
```

### Fonctions renvoyant un objet

Une fonction peut également renvoyer un objet en spécifiant le type de l'objet comme type de retour.

Exemple

Voici un exemple de fonction qui renvoie un objet représentant une personne :

```
function getPerson(): { name: string, age: number } {
  return {
    name: "Alice",
    age: 30
  };
}
```

Dans cet exemple, la fonction `getPerson` renvoie un objet avec 2 propriétés : « *name* » de type `string` et « *age* » de type `number`. Nous utilisons un type d'objet littéral pour spécifier le type de retour de la fonction.

Tous droits réservés à STUDI - Reproduction interdite

Nous pouvons appeler cette fonction et stocker le résultat dans une variable :

```
let person = getPerson(); // person vaut { name: "Alice", age: 30 }
```

### Fonctions renvoyant une fonction

Enfin, une fonction peut renvoyer une autre fonction en spécifiant le type de la fonction comme type de retour.

Exemple

```
function getMultiplier(factor: number): (x: number) => number {
  return function(x: number) {
    return x * factor;
  };
}
```

Dans cet exemple, la fonction `getMultiplier` prend un paramètre « *factor* » de type number et renvoie une fonction qui prend un paramètre « *x* » de type `number` et renvoie le produit de « *x* » et de « *factor* ». Nous utilisons le type `(x: number) => number` pour spécifier que la fonction renvoyée prend un paramètre de type `number` et renvoie un résultat de type `number`.

Nous pouvons appeler cette fonction et stocker le résultat dans une variable :

```
let multiplyBy2 = getMultiplier(2); // multiplyBy2 est une fonction qui multiplie par 2
let result = multiplyBy2(3); // result vaut 6
```

Dans cet exemple, nous appelons la fonction `getMultiplier` avec un argument de 2, ce qui renvoie une fonction qui multiplie par 2. Nous stockons cette fonction dans la variable `multiplyBy2`. Nous appelons ensuite cette fonction avec un argument de 3, ce qui renvoie 6.

### Fonctions anonymes

En TypeScript, il est possible de définir des fonctions anonymes, qui sont des variantes de la syntaxe traditionnelle des fonctions.

Une fonction anonyme est une fonction qui n'a pas de nom. Elle est définie comme une expression et peut être stockée dans une variable ou passée comme argument à une autre fonction.

Exemple

Voici un exemple de fonction anonyme qui prend deux paramètres a et b de type number et renvoie leur somme :

```
let sum = function(a: number, b: number): number {
  return a + b;
};
```

Dans cet exemple, nous définissons une fonction anonyme en utilisant la `syntaxe function(a: number, b: number): number`. Nous stockons cette fonction dans la variable `sum`. Cette fonction prend 2 paramètres a et b de type `number` et renvoie leur somme de type `number`.

Nous pouvons appeler cette fonction en utilisant le nom de la variable qui la contient :

```
let result = sum(3, 4); // result vaut 7
```


Tous droits réservés à STUDI - Reproduction interdite

Les fonctions anonymes peuvent être utiles dans des situations où nous avons besoin d'utiliser la fonction seulement une fois. Cela rend le code plus lisible en évitant d'avoir une fonction définie quelque part dans le code qui n'est utilisée qu'une seule fois. En outre, les fonctions anonymes peuvent être utilisées comme arguments de fonctions, ce qui peut être pratique pour des opérations comme le tri ou le filtrage de tableaux.

Cependant, si les fonctions sont trop complexes ou imbriquées. Il est préférable de définir des fonctions nommées séparées pour une meilleure lisibilité.

### Fonctions fléchées

En TypeScript, il est aussi possible de définir des fonctions fléchées, qui sont également des variantes de la syntaxe traditionnelle des fonctions.

Une fonction fléchée est une version plus concise de la syntaxe des fonctions. Elle utilise une flèche (=>) pour séparer les paramètres de la fonction de son corps.

Exemple

Voici un exemple de fonction fléchée qui prend 2 paramètres a et b de type `number` et renvoie leur somme :

```
let sum = (a: number, b: number): number => a + b;
```

Dans cet exemple, nous définissons une fonction fléchée en utilisant la syntaxe `(a: number, b: number): number => a + b`. Cette fonction prend 2 paramètres a et b de type `number` et renvoie leur somme de type `number`.

Nous pouvons appeler cette fonction de la même manière qu'une fonction anonyme :

```
let result = sum(3, 4); // result vaut 7
```

Les fonctions fléchées peuvent également être utilisées pour des fonctions à un seul paramètre :

```
let double = (x: number) => x * 2;
```

Dans cet exemple, nous définissons une fonction fléchée qui prend un paramètre x de type `number` et renvoie le double de x.

Il est recommandé d'utiliser les fonctions fléchées lorsque la fonction ne nécessite pas d'accéder à `this`. Cela est dû au fait que les fonctions fléchées capturent automatiquement la valeur de `this` de l'environnement dans lequel elles ont été créées, contrairement aux fonctions ordinaires qui ont leur propre valeur de `this`. De plus, les fonctions fléchées sont souvent plus courtes et plus lisibles que les fonctions ordinaires, ce qui les rend très utiles pour les fonctions de rappel ou les fonctions qui retournent une fonction.

### Fonctions anonymes et fonctions fléchées

Exemple

Voici un exemple qui utilise des fonctions anonymes et des fonctions fléchées :

```
let numbers = [1, 2, 3, 4, 5];
// Utilisation d'une fonction anonyme pour filtrer les nombres pairs
let evenNumbers = numbers.filter(function(n) {
  return n % 2 == 0;
});
// Utilisation d'une fonction fléchée pour calculer le carré de chaque nombre
let squaredNumbers = numbers.map(n => n * n);
console.log(evenNumbers);     // Affiche [2, 4]
console.log(squaredNumbers);  // Affiche [1, 4, 9, 16, 25]
```

Tous droits réservés à STUDI - Reproduction interdite

Dans cet exemple, nous avons un tableau de nombres que nous utilisons pour illustrer l'utilisation de fonctions anonymes et de fonctions fléchées.

La première ligne utilise la méthode `filter` pour filtrer les nombres pairs du tableau `numbers`. Nous utilisons une fonction anonyme pour définir le critère de filtrage. Cette fonction prend un paramètre « *n* » qui représente chaque élément du tableau et renvoie un booléen indiquant si « *n* » est pair ou non.

La deuxième ligne utilise la méthode `map` pour calculer le carré de chaque nombre dans le tableau `numbers`. Nous utilisons une fonction fléchée pour définir la transformation à appliquer à chaque élément du tableau. Cette fonction prend un paramètre « *n* » qui représente chaque élément du tableau et renvoie le carré de « *n* ».

Enfin, nous utilisons la fonction `console.log` pour afficher les tableaux `evenNumbers` et `squaredNumbers` dans la console.

Cet exemple montre comment les fonctions anonymes et les fonctions fléchées peuvent être utilisées pour effectuer des opérations sur des tableaux en TypeScript de manière concise et lisible.

### Fonctions génériques

En TypeScript, les fonctions génériques sont des fonctions qui peuvent travailler avec différents types de données sans connaître à l'avance ces types. Cela permet de réutiliser le code de manière plus flexible et d'éviter les répétitions.

Pour définir une fonction générique, on utilise le mot-clé `function` suivi d'un nom de fonction et des chevrons <> contenant le ou les types génériques. Ces types génériques sont utilisés pour définir les paramètres et le type de retour de la fonction.

Exemple

Voici un exemple de fonction générique qui retourne un tableau inversé :

```
function inverserTableau<T>(tableau: T[]): T[] {
  return tableau.reverse();
}
```

Dans cet exemple, le type générique « *T* » est utilisé pour définir le type des éléments du tableau passé en paramètre et le type de retour de la fonction. La méthode `reverse()` est appliquée sur le tableau, qui est ensuite retourné.

On peut ensuite utiliser cette fonction avec différents types de données, par exemple :

```
const tableau1 = [1, 2, 3, 4, 5]; // Tableau de nombre
const tableau2 = ["a", "b", "c", "d", "e"]; // Tableau de string
console.log(inverserTableau(tableau1)); // [5, 4, 3, 2, 1]
console.log(inverserTableau(tableau2)); // ["e", "d", "c", "b", "a"]
```

Dans cet exemple, la fonction `inverserTableau()` est appelée avec 2 tableaux de types différents, un tableau de nombres et un tableau de chaînes de caractères.

Les fonctions génériques permettent ainsi de rendre le code plus flexible et plus réutilisable en évitant les répétitions de code pour différentes combinaisons de types de données.

## B. Exercice : Quiz

[solution n°2 p. 26]

### Question 1

Comment définir le type de retour d'une fonction en TypeScript ?


Tous droits réservés à STUDI - Reproduction interdite

En utilisant le mot-clé « *return* » suivi du type de retour

En utilisant une annotation de type après les parenthèses de la fonction

En utilisant l'instruction « => » pour définir une fonction fléchée.

## Question 2

Comment définir un type de paramètre optionnel dans une fonction en TypeScript ?

En ajoutant un point d'interrogation après le nom du paramètre

En ajoutant le mot-clé « *optional* » avant le nom du paramètre

En utilisant un point d'interrogation après le nom du paramètre et en initialisant le paramètre à « *undefined* »

## Question 3

Pourquoi utiliser une fonction anonyme ?

Lorsqu'on a besoin d'utiliser la fonction une seule fois

Pour éviter de polluer l'espace de noms global avec une fonction qui ne sera utilisée qu'une seule fois

Pour la possibilité de l'utiliser comme argument dans d'autres fonctions

Toutes les réponses ci-dessus

## Question 4

Quand utiliser une fonction fléchée en TypeScript ?

**A** Lorsque la fonction ne nécessite qu'un seul paramètre

**B** Lorsque la fonction ne nécessite pas d'accéder à « *this* »

**C** Lorsque la fonction est courte et simple

## Question 5

Comment utilisez-vous une fonction fléchée pour définir une fonction qui prend 2 paramètres x et y de type « *number* » et qui renvoie leur somme ?

```
let maFonction = (x: number, y: number) => x + y;
```

Tous droits réservés à STUDI - Reproduction interdite

```
function maFonction(x: number, y: number) { return x + y; }
```

```
let maFonction(x: number, y: number) => { return x + y; }
```


Tous droits réservés à STUDI - Reproduction interdite

# IV Le casting

## A. Le casting

Az Définition

Le casting est une opération de conversion de type de données d'une variable en un autre type de données. En TypeScript, comme dans de nombreux autres langages de programmation, il est possible d'effectuer des opérations de casting sur les variables.

Il existe 2 types de castings en TypeScript : le casting **implicite** et le casting **explicite**.

### Le casting implicite

Le casting implicite est automatiquement effectué par le compilateur lorsqu'il est possible de convertir implicitement un type en un autre type. Par exemple, si vous ajoutez une chaîne de caractères à un nombre, TypeScript convertit automatiquement le nombre en une chaîne de caractères pour concaténer les 2.

Exemple de casting implicite :

```
let a: number = 5;
let b: string = "10";
let c: string = a + b; // TypeScript convertit automatiquement 'a' en une chaîne de caractères pour concaténer les deux
console.log(c); // Affiche '510'
```

### Le casting explicite

Le casting explicite, en revanche, nécessite que le développeur spécifie explicitement le type de données dans lequel il souhaite convertir une variable. Cela se fait en utilisant l'opérateur `as` ou en utilisant des crochets d'angle <>.

Exemple de casting explicite avec l'opérateur `as` :

```
 let a: any = "5";
let b: number = a as number; // convertit explicitement 'a' en un nombre en utilisant l'opérateur "as"
console.log(b); // Affiche 5
```

Exemple de casting explicite avec des crochets d'angle <> :

```
let a: any = "5";
let b: number = <number>a; // converti 'a' en un nombre en utilisant des crochets d'angle < >
console.log(b); // Affiche 5
```

Il est important de noter que le casting explicite peut entraîner des erreurs s'il est utilisé de manière incorrecte ou si le type de données d'origine ne peut pas être converti en un autre type de données. Il est donc conseillé de l'utiliser avec précaution et de vérifier que les types de données sont compatibles avant d'effectuer un casting.

Tous droits réservés à STUDI - Reproduction interdite

### Le casting implicite et explicite

Exemple

Voici un exemple qui illustre l'utilisation du casting implicite et explicite avec des tableaux et des fonctions en TypeScript.

Supposons que nous ayons un tableau de nombres et que nous voulions appliquer une fonction qui multiplie chaque nombre par un facteur donné. Cependant, la fonction que nous voulons utiliser prend en entrée un tableau de nombres sous forme de chaîne de caractères et renvoie un tableau de nombres sous forme de chaîne de caractères.

Pour résoudre ce problème, nous devons utiliser le casting explicite pour convertir le tableau de nombres en tableau de chaînes de caractères avant d'appeler la fonction. Puis utiliser le casting implicite pour convertir le tableau de chaînes de caractères résultant en un tableau de nombres.

Voici le code correspondant :

```
// Tableau de nombres
const nombres = [1, 2, 3, 4, 5];
// Fonction qui multiplie chaque nombre par un facteur donné
function multiplierNombres(nombres: string[], facteur: number): string[] {
  const resultat: string[] = [];
  nombres.forEach(nombre => {
    const resultatMultiplication = Number(nombre) * facteur;
    resultat.push(resultatMultiplication.toString());
  });
  return resultat;
}
// Appel de la fonction avec le tableau de nombres
const facteur = 2;
const nombresEnChaineDeCaracteres = nombres.map(nombre => nombre.toString());
const resultatsMultiplication = multiplierNombres(nombresEnChaineDeCaracteres as string[], facteur);
const resultats = resultatsMultiplication.map(resultat => Number(resultat)) as number[];
// Affichage des résultats
console.log(nombres); // [1, 2, 3, 4, 5]
console.log(resultats); // [2, 4, 6, 8, 10]
```

Dans cet exemple, nous avons d'abord converti le tableau de nombres en un tableau de chaînes de caractères en utilisant la méthode `map` et la méthode `toString`. Nous avons ensuite appelé la fonction `multiplierNombres` avec le tableau de chaînes de caractères en utilisant le casting explicite `as string[]`.

La fonction a renvoyé un tableau de chaînes de caractères représentant les résultats de la multiplication, que nous avons ensuite converti en un tableau de nombres en utilisant la méthode `map` et la méthode `Number`, suivie du casting implicite `as number[]`.

En résumé, le casting explicite et le casting implicite sont des techniques utiles pour manipuler des données de différents types en TypeScript, mais il est important de les utiliser avec précaution pour éviter les erreurs de type.

## B. Exercice : Quiz

[solution n°3 p. 27]

### Question 1

Quel est le résultat de la conversion implicite suivante en TypeScript ?

```
let a : number = 7;
let b : any = "9";
let c : string = a + b;
console.log(c);
```



Tous droits réservés à STUDI - Reproduction interdite

16

79

Erreur de typage

## Question 2

Comment réaliser un casting explicite vers un type d'énumération en TypeScript ?

En utilisant l'opérateur « *as* »

En utilisant la méthode « *valueOf()* »

En utilisant l'opérateur « *!* »

## Question 3

On caste la variable « *any* » x en un number, en utilisant le mot-clé « *as* » :

```
let x : any = "5";
console.log((...) + 10); // Résultat 15
```

Remplacez les 3 points par :

x as unknown

number as x

x as number

parseInt(x)

## Question 4

On caste la variable « *unknown* » myVar en un string, en utilisant les crochets d'angle <> :

```
let myVar: unknown = "Hello world!";
console.log((...).length);
```

Remplacez les 3 points par :

<myVar>string

<string>myVar

myVar<string>

Tous droits réservés à STUDI - Reproduction interdite

## Question 5

Quel est le résultat de la conversion implicite suivante en TypeScript ?

```
let x: any[] = ["5", "10", "15"];
let y: number[] = x.map(Number);
console.log(y); // Le résultat = [ 5, 10, 15 ] ou [ '5', '10', '15' ] ou undefined
```

« *y* » est maintenant un tableau de type number avec les valeurs 5, 10 et 15

« *y* » est maintenant un tableau de type string avec les valeurs 5, 10 et 15

L'opération de conversion échouera et y sera « *undefined* »


Tous droits réservés à STUDI - Reproduction interdite

# V Essentiel

Les tableaux en TypeScript peuvent être typés avec des types de données spécifiques en utilisant la syntaxe `type[]` ou `Array<type>`. On peut manipuler les tableaux avec des méthodes telles que `map`, `filter` et `reduce`.

Les tuples en TypeScript permettent de définir des tableaux avec un nombre fixe d'éléments de types différents. Les éléments individuels peuvent être typés avec des types différents et l'ordre des éléments dans le tuple est important. On peut manipuler les tuples avec des méthodes telles que `push`, `pop` et `slice`.

Les énumérations en TypeScript permettent de définir un ensemble de valeurs numériques ou chaînes de caractères associées à des noms symboliques, ce qui facilite la lecture et la compréhension du code. Les membres d'une énumération peuvent être des nombres ou des chaînes de caractères et peuvent être utilisés dans des expressions et des instructions switch.

Les fonctions en TypeScript peuvent prendre des paramètres typés et des types de retour, ce qui permet de mieux documenter le code et de détecter les erreurs de typage plus tôt. On peut aussi définir des paramètres optionnels et des paramètres avec des valeurs par défaut.

Les fonctions en TypeScript peuvent également être des fonctions anonymes ou des fonctions fléchées, qui offrent une syntaxe plus concise pour définir des fonctions simples.

Tous droits réservés à STUDI - Reproduction interdite

# VI Auto-évaluation

## A. Exercice

Vous êtes développeur et travaillez pour une entreprise de développement de logiciels depuis un certain temps. Récemment, deux missions vous ont été attribuées dans le cadre de deux projets différents.

### Question 1

[solution n°4 p. 29]

Déclarez une énumération nommée « *Jours* » qui contient les valeurs « *Lundi* », « *Mardi* », « *Mercredi* », « *Jeudi* », « *Vendredi* », « *Samedi* » et « *Dimanche* ».

Créez une fonction nommée « *obtenirJour* » qui prend un paramètre de type nombre et retourne le jour correspondant dans l'énumération « *Jours* » (par exemple, si le nombre est 1, la fonction doit retourner « *Lundi* »).

### Question 2

[solution n°5 p. 30]

Créez une fonction qui prendra en paramètre 2 nombres : le coût de fabrication d'un objet et son prix de vente. La fonction affichera le bénéfice ou la perte en fonction du prix de vente.

## B. Test

### Exercice 1 : Quiz

[solution n°6 p. 30]

#### Question 1

Quelle est la syntaxe pour déclarer un tableau de type générique, qui contient des strings et des nombres en TypeScript ?

- let myArray <string | number>;
- let myArray : Array<string | number>;
- let myArray : [string, number];

#### Question 2

Quelle est la syntaxe pour déclarer une fonction qui prend 2 paramètres de type string et retourne un type string en TypeScript ?

- function myFunction(str1: string, str2: string){
- function myFunction(str1: number, str2: string): string {
- function myFunction(str1: string, str2: string): string {

#### Question 3

Comment définir une fonction fléchée qui prend un paramètre « *x* » de type number et qui renvoie le carré de « *x* » ?


Tous droits réservés à STUDI - Reproduction interdite

let maFonction = function(x: number) { return x * x; };

let maFonction = (x: number) => x * x;

let maFonction(x: number) { return x * x; }

## Question 4

Soit le code suivant :

```
1 let maValeur: any = 5;
```

Effectuez un cast de `maValeur` en type string et stockez le résultat dans une variable nommée « *maChaine* ». Choisissez la bonne réponse :

let maChaine: string = <string>maValeur;

let maChaine: string = maValeur as string;

Les 2 réponses sont correctes

## Question 5

Comment définir une fonction générique qui prend un tableau de type « *T* » et un élément de type « *T* » et qui renvoie un tableau avec l'élément ajouté ?

function ajouterElement(tableau: any[], element: any): any[] { ... }

function ajouterElement(tableau: [], element: any): [] { ... }

function ajouterElement<T>(tableau: T[], element: T): T[] { ... }

Tous droits réservés à STUDI - Reproduction interdite

# Solutions des exercices

## Solution n°1

[exercice p. 8]

**Question 1**

Quelle est la syntaxe pour déclarer un tableau de nombres en TypeScript ?

let arr: array = [1, 2, 3];

✓ let arr: number[] = [1, 2, 3];

let arr = [1, 2, 3];

En TypeScript, on utilise la syntaxe `let nomDuTableau: type[] = [valeur1, valeur2, valeur3, ...]` pour déclarer un tableau de valeurs d'un type spécifié. Dans ce cas, la syntaxe `let arr: number[] = [1, 2, 3];` déclare un tableau de nombres avec les valeurs 1, 2 et 3. La réponse 1 est incorrecte, car « *array* » n'est pas un type valide en TypeScript, tandis que la réponse 3 ne spécifie pas explicitement le type du tableau.

**Question 2**

Quelle est la différence entre un tableau et un tuple en TypeScript ?

✓ Un tableau est une collection de valeurs de même type tandis qu'un tuple est une collection de valeurs de types différents/égaux

Il n'y a pas de différence significative entre un tableau et un tuple en TypeScript

Un tableau est une collection de valeurs de types différents tandis qu'un tuple est une collection de valeurs de même type

En TypeScript, un tableau est une collection ordonnée d'éléments du même type qui peut être modifiée en ajoutant ou en supprimant des éléments. Un tuple, en revanche, est également une collection ordonnée d'éléments, mais chaque élément peut avoir une collection de valeurs de types différents/égaux

**Question 3**

L'ordre des types de valeurs dans un tuple n'est pas important.



Tous droits réservés à STUDI - Reproduction interdite

Vrai

✓ Faux

L'ordre des types de valeurs dans un tuple est important, car il détermine l'ordre dans lequel les valeurs doivent être assignées ou retournées.

**Question 4**

Quelle est la syntaxe pour déclarer un tuple qui contient une chaîne de caractères et un nombre en TypeScript ?

let myTuple: [string, number] = [1, "Hello"];

let myTuple: [number, string] = ["Hello", 1];

✓ let myTuple: [string, number] = ["Hello", 1];

La réponse est : `let myTuple: [string, number] = ["Hello", 1];`, car il faut respecter l'ordre des types définis entre crochets lors de la déclaration d'un tuple.

**Question 5**

Quelle est la syntaxe pour déclarer une énumération en TypeScript ?

✓ enum Seasons{Spring, Summer, Autumn, Winter};

enum Seasons = {Spring, Summer, Autumn, Winter};

let Seasons = enum {Spring, Summer, Autumn, Winter};

La réponse correcte est : `enum Seasons{Spring, Summer, Autumn, Winter};`. Dans cet exemple, nous déclarons une énumération appelée « Seasons » qui contient 7 valeurs : « *Spring* », « *Summer* », « *Autumn* », « *Winter* ». Les valeurs numériques correspondantes seront automatiquement assignées à partir de 0. Par exemple, « *Spring* » aura la valeur 0, « *Summer* » aura la valeur 1, « *Autumn* » aura la valeur 2 et « *Winter* » aura la valeur 3.

Tous droits réservés à STUDI - Reproduction interdite

# Solution n°2

[exercice p. 14]

**Question 1**

Comment définir le type de retour d'une fonction en TypeScript ?

- En utilisant le mot-clé « *return* » suivi du type de retour
- ✓ En utilisant une annotation de type après les parenthèses de la fonction
- En utilisant l'instruction « => » pour définir une fonction fléchée.

Pour définir le type de retour d'une fonction en TypeScript, on utilise une annotation de type après les parenthèses de la fonction. Par exemple, pour définir une fonction qui renvoie un nombre, on écrirait `function maFonction(): number { ... }`.

**Question 2**

Comment définir un type de paramètre optionnel dans une fonction en TypeScript ?

- ✓ En ajoutant un point d'interrogation après le nom du paramètre
- En ajoutant le mot-clé « *optional* » avant le nom du paramètre
- En utilisant un point d'interrogation après le nom du paramètre et en initialisant le paramètre à « *undefined* »

Pour définir un type de paramètre optionnel dans une fonction en TypeScript, on ajoute un point d'interrogation après le nom du paramètre. Par exemple, pour définir une fonction avec un paramètre optionnel x, on écrirait `function maFonction(x?: number) { ... }`. On peut également initialiser le paramètre à « *undefined* » en utilisant `x = undefined`.

**Question 3**

Pourquoi utiliser une fonction anonyme ?

- Lorsqu'on a besoin d'utiliser la fonction une seule fois
- Pour éviter de polluer l'espace de noms global avec une fonction qui ne sera utilisée qu'une seule fois
- Pour la possibilité de l'utiliser comme argument dans d'autres fonctions
- ✓ Toutes les réponses ci-dessus



Tous droits réservés à STUDI - Reproduction interdite

Les fonctions anonymes peuvent être utiles dans des situations où nous avons besoin d'utiliser la fonction une seule fois, ou pour éviter de polluer l'espace de noms global avec une fonction qui ne sera utilisée qu'une seule fois. Il est aussi possible de l'utiliser comme argument dans d'autres fonctions.

**Question 4**

Quand utiliser une fonction fléchée en TypeScript ?

- **A** Lorsque la fonction ne nécessite qu'un seul paramètre
- **B** Lorsque la fonction ne nécessite pas d'accéder à « *this* »
- **C** Lorsque la fonction est courte et simple

Les fonctions fléchées sont idéales pour des fonctions courtes, des callbacks, ou lorsque vous souhaitez conserver le contexte lexical de this. Cependant, pour des fonctions plus complexes ou lorsque vous avez besoin de fonctionnalités spécifiques comme arguments ou prototype, les fonctions classiques peuvent être plus appropriées.

**Question 5**

Comment utilisez-vous une fonction fléchée pour définir une fonction qui prend 2 paramètres x et y de type « *number* » et qui renvoie leur somme ?

- ✓ let maFonction = (x: number, y: number) => x + y;
- function maFonction(x: number, y: number) { return x + y; }
- let maFonction(x: number, y: number) => { return x + y; }

Pour définir une fonction fléchée en TypeScript, on utilise la `syntaxe (param1: type, param2: type) => expression`. Dans ce cas, la fonction prend 2 paramètres x et y de type « *number* » et renvoie leur somme. La syntaxe correcte pour définir une telle fonction fléchée serait donc la suivante : `let maFonction = (x: number, y: number) => x + y;`.

# Solution n°3

[exercice p. 18]

**Question 1**

Quel est le résultat de la conversion implicite suivante en TypeScript ?

```
let a : number = 7;
let b : any = "9";
let c : string = a + b;
console.log(c);
```

Tous droits réservés à STUDI - Reproduction interdite

16

✓ 79

Erreur de typage

Le résultat de la conversion implicite suivante en TypeScript serait « *79* ». Cela est dû au fait que le nombre 7 est converti implicitement en chaîne de caractère lors de l'opération d'addition avec la variable « *b* » de type any.

**Question 2**

Comment réaliser un casting explicite vers un type d'énumération en TypeScript ?

✓ En utilisant l'opérateur « *as* »

En utilisant la méthode « *valueOf()* »

En utilisant l'opérateur « *!* »

On utilise l'opérateur « *as* » pour réaliser un casting explicite vers un type d'énumération en TypeScript.

**Question 3**

On caste la variable « *any* » x en un number, en utilisant le mot-clé « *as* » :

```
let x : any = "5";
console.log((...) + 10); // Résultat 15
```

Remplacez les 3 points par :

x as unknown

number as x

✓ x as number

parseInt(x)

Dans ce cas, nous avons utilisé le mot-clé « *as* » pour effectuer un casting explicite de la variable « *x* » en tant que number. Nous avons également utilisé les parenthèses pour encadrer l'expression (x as number), de manière à ce que le compilateur comprenne que nous voulons effectuer l'addition entre un nombre et la valeur 10. Ainsi, le résultat affiché dans la console sera bien 15.


Tous droits réservés à STUDI - Reproduction interdite

**Question 4**

On caste la variable « *unknown* » myVar en un string, en utilisant les crochets d'angle <> :

```
let myVar: unknown = "Hello world!";
console.log((...).length);
```

Remplacez les 3 points par :

- <myVar>string
- ✓ <string>myVar
- myVar<string>

On peut utiliser l'opérateur de casting explicite `<string>` pour convertir la variable `myVar` de type « *unknown* » en type « *string* ». Ensuite, on peut accéder à la propriété « *length* » de la chaîne de caractères en utilisant la notation avec point « . ». La bonne syntaxe serait donc `(myVar as string).length`.

**Question 5**

Quel est le résultat de la conversion implicite suivante en TypeScript ?

```
let x: any[] = ["5", "10", "15"];
let y: number[] = x.map(Number);
console.log(y); // Le résultat = [ 5, 10, 15 ] ou [ '5', '10', '15' ] ou undefined
```

- ✓ « *y* » est maintenant un tableau de type number avec les valeurs 5, 10 et 15
- « *y* » est maintenant un tableau de type string avec les valeurs 5, 10 et 15
- L'opération de conversion échouera et y sera « *undefined* »

« *y* » est maintenant un tableau de type number avec les valeurs 5, 10 et 15, car la méthode map a appliqué la fonction Number à chaque élément du tableau.

[exercice p. 22]

## Solution n°4

Cet exercice utilise une énumération appelée « *Jours* » qui définit les différents jours de la semaine. La fonction « *obtenirJour* » prend en paramètre un nombre qui correspond à un jour de la semaine et renvoie le nom du jour correspondant.

```
// Déclaration énumération nommée Jours
enum Jours {
Lundi = 1,
Mardi,
Mercredi,
Jeudi,
Vendredi,
Samedi,
```

Tous droits réservés à STUDI - Reproduction interdite

```
Dimanche
}
// Création fonction nommée obtenirJour
function obtenirJour(nombre: number): string {
  return Jours[nombre];
}
console.log(obtenirJour(1)); // Lundi
```

## [exercice p. 22] Solution n°5

La fonction bénéfice prend 2 paramètres : le prix de vente et le coût de fabrication. Elle calcule le bénéfice ou la perte, puis l'affiche dans la console. La fonction utilise des conditions pour déterminer si le résultat est positif, négatif ou nul. La valeur de retour de la fonction est `void`, car elle n'a pas besoin de renvoyer une valeur à l'appelant.

```
function benef(prixVt: number, cFab: number): void {
  let benef: number = prixVt - cFab;
  if (benef > 0) {
    console.log(`Le bénéfice est de : ${benef} €`);
  } else if (benef < 0) {
    console.log(`Les pertes sont de : ${benef} €`);
  } else {
    console.log(`Pas de bénéfice : ${benef} €`);
  }
}
benef(167, 167); // affiche "Pas de bénéfice : 0 €"
```

## Solution n°6

[exercice p. 22]

**Question 1**

Quelle est la syntaxe pour déclarer un tableau de type générique, qui contient des strings et des nombres en TypeScript ?

let myArray <string | number>;

✓ let myArray : Array<string | number>;

let myArray : [string, number];

On utilise syntaxe `Array<type>` pour déclarer un tableau de type générique, et on utilise l'union de types `string | number` pour indiquer que le tableau peut contenir des éléments de type string ou number. Donc la réponse exacte est `let myArray : Array<string | number>;`.

**Question 2**

Quelle est la syntaxe pour déclarer une fonction qui prend 2 paramètres de type string et retourne un type string en TypeScript ?


Tous droits réservés à STUDI - Reproduction interdite

function myFunction(str1: string, str2: string){

function myFunction(str1: number, str2: string): string {

✓ function myFunction(str1: string, str2: string): string {

La syntaxe pour déclarer une fonction qui prend 2 paramètres de type string et retourne un type string en TypeScript est `function myFunction(str1: string, str2: string): string {`. Ici, « *str1* » et « *str2* » sont les noms des paramètres de la fonction.

**Question 3**

Comment définir une fonction fléchée qui prend un paramètre « *x* » de type number et qui renvoie le carré de « *x* » ?

let maFonction = function(x: number) { return x * x; };

✓ let maFonction = (x: number) => x * x;

let maFonction(x: number) { return x * x; }

Pour définir une fonction fléchée en TypeScript, on utilise la syntaxe `(param1: type, param2: type) => { ... }` ou `(param1: type, param2: type) => expression`. Dans ce cas, la fonction prend un paramètre « *x* » de type number et renvoie le carré de « *x* ».

**Question 4**

Soit le code suivant :

```
1 let maValeur: any = 5;
```

Effectuez un cast de `maValeur` en type string et stockez le résultat dans une variable nommée « *maChaine* ». Choisissez la bonne réponse :

let maChaine: string = <string>maValeur;

let maChaine: string = maValeur as string;

✓ Les 2 réponses sont correctes

Les deux réponses sont correctes pour effectuer une conversion de type (ou « *casting* ») en TypeScript en utilisant la syntaxe `<type>` ou l'opérateur `as`. En effet, elles sont équivalentes et donnent le même résultat dans ce cas précis.

Tous droits réservés à STUDI - Reproduction interdite

**Question 5**

Comment définir une fonction générique qui prend un tableau de type « *T* » et un élément de type « *T* » et qui renvoie un tableau avec l'élément ajouté ?

function ajouterElement(tableau: any[], element: any): any[] { ... }

function ajouterElement(tableau: [], element: any): [] { ... }

✓ function ajouterElement<T>(tableau: T[], element: T): T[] { ... }

Pour définir une fonction générique en TypeScript, on utilise le symbole <T> après le nom de la fonction pour indiquer que la fonction prend un type générique « *T* ». Dans ce cas, la fonction prend un tableau de type « *T* » et un élément de type « *T* » et renvoie un tableau de type « *T* » qui contient l'élément ajouté.


Tous droits réservés à STUDI - Reproduction interdite